

Nombre de la materia

CALIDAD EN SOFTWARE II.

Nombre del programa

Ingeniería de software

Nombre del estudiante

MATÍAS VIRGÜEZ

Nombre de la actividad

Script y ejecución de prueba de carga.

Unidad #

Unidad 2 actividad 3

Nombre del profesor

JEFFERSON URREGO GUERRERO.

Fecha

19-09-2026

Prueba de rendimiento

Sistema bajo prueba: QuickPizza (quickpizza.grafana.com)

Calidad en Software II — EV-4 / EV-5 — Prueba de rendimiento (carga, estrés, resistencia)

LINK REPOSITORIO:

<https://github.com/Matiasv18/prueba-rendimiento-quickpizza.git>

1. Introducción

En este informe se presenta el diseño, ejecución y análisis de una prueba de rendimiento realizada sobre el servicio público QuickPizza, proporcionado por Grafana Labs para realizar pruebas de carga con k6. Se realizaron tres tipos de pruebas: carga, estrés y resistencia (soak), utilizando dos rutas representativas del sistema: la página principal (GET /) y el endpoint de recomendación de pizza (POST /api/pizza), el cual incluye lógica de negocio en el servidor. El objetivo principal fue observar el comportamiento del sistema bajo diferentes niveles de carga, compararlo con los valores definidos en el SLA/SLO y encontrar posibles puntos de mejora o cuellos de botella.

2. Modelo de carga

Para la prueba se planteó un escenario basado en un flujo de usuario sencillo y representativo. El usuario ingresa primero a la página principal y posteriormente realiza una solicitud para obtener una recomendación de pizza. Para esta solicitud se utilizaron ocho perfiles diferentes, combinando restricciones de calorías entre 300 y 700 por porción y la preferencia por opciones vegetarianas. También se agregó un tiempo de espera (think time) aleatorio entre 1 y 4 segundos entre las acciones, buscando que el comportamiento fuera más cercano al de un usuario real y no generara solicitudes continuas de forma artificial.

Se definieron tres escenarios de prueba. El primero fue una prueba de carga con 20 usuarios virtuales (VUs) durante 6 minutos, con el propósito de revisar el comportamiento del sistema bajo una carga esperada. El segundo correspondió a una prueba de estrés, aumentando progresivamente la cantidad de usuarios de 10 en 10 hasta llegar a 100 VUs, con el fin de identificar en qué punto comienza a presentarse una degradación del rendimiento. Finalmente, se realizó una prueba de resistencia con 15 VUs durante 30 minutos para observar si el sistema mantenía un comportamiento estable durante un periodo prolongado y detectar posibles problemas de degradación.

Como referencia para el análisis se establecieron los siguientes objetivos de SLA/SLO: un P95 menor a 800 ms para GET /, un P95 menor a 2000 ms para POST /api/pizza, una tasa de errores inferior al 1 % y un throughput sostenido de al menos 15 solicitudes por segundo.

3. Metodología

El script de pruebas fue desarrollado en k6 utilizando JavaScript. Los datos utilizados durante la ejecución fueron parametrizados mediante un archivo externo (data/users.json), evitando dejar valores fijos directamente dentro del código y facilitando la repetición de las pruebas con diferentes datos. Además, se configuraron checks para validar los códigos de respuesta y el contenido de las respuestas obtenidas en el endpoint de recomendación.

También se definieron thresholds para los diferentes endpoints, tomando como referencia los valores establecidos en el SLA/SLO. De esta manera, los criterios de rendimiento quedaron integrados directamente en el script y pudieron ser comprobados durante cada ejecución.

Para conservar las evidencias y facilitar la reproducibilidad, cada prueba generó resultados en formatos CSV y JSON, además de un reporte HTML mediante la librería k6-reporter. Como parte de la observabilidad, durante los momentos más importantes de cada prueba se realizó seguimiento al consumo de CPU y memoria mediante el Administrador de tareas de Windows. Estas métricas permitieron complementar los resultados obtenidos por k6 y tener una referencia del comportamiento de los recursos del equipo durante las pruebas.

4. Resultados

La siguiente tabla resume las métricas clave obtenidas en las tres ejecuciones:

Tabla 1 Métricas obtenidas en las ejecuciones. Fuente (El Autor).

Prueba	VUs	Duración	P95 home	P95 pizza	Error %	Throughput
Carga	20	6 min	104.58 ms	659.59 ms	0.00%	7.60 req/s
Estrés	hasta 100	8 min	101.17 ms	672.54 ms	0.00%	18.43 req/s
Resistencia	15	30 min	102.29 ms	686.84 ms	0.00%	6.13 req/s

En las tres pruebas se procesaron en total 22.746 solicitudes, sin registrar errores, y se obtuvieron 34.119 checks aprobados. La latencia P95 del endpoint de recomendación se mantuvo entre 659.59 ms y 686.84 ms, incluso cuando la prueba de estrés llegó hasta 100 VUs. Esto muestra que, dentro del rango evaluado, el aumento de usuarios no produjo una degradación importante en los tiempos de respuesta.

Durante la prueba de resistencia también se revisó el comportamiento del equipo utilizado como generador de carga. Se tomaron capturas en los minutos 0, 15 y 30 para observar CPU, memoria y red.

Tabla 2 capturas de pantalla comportamiento CPU vs memoria vs Red. Fuente (El Autor).

Momento	CPU	Memoria	Red
Inicio (min 0)	54%	94%	2%
Mitad (min 15)	29%	89%	0%
Final (min 30)	13%	92%	0%

Los datos muestran que el consumo de CPU del generador disminuyó durante la prueba, mientras que la memoria se mantuvo entre 89% y 94%, sin presentar un aumento progresivo. Por lo tanto, no se observaron señales de una fuga de memoria en el cliente utilizado para generar la carga durante los 30 minutos.

5. Contraste con el SLA/SLO

Al comparar los resultados con los objetivos establecidos, la latencia y la tasa de errores cumplieron los valores definidos en las tres pruebas. El P95 de GET / se mantuvo por debajo de 800 ms y el P95 de POST /api/pizza estuvo por debajo de 2000 ms. Además, la tasa de errores fue de 0.00% en todos los escenarios. El throughput presentó una diferencia frente al objetivo de 15 req/s. En la prueba de estrés se alcanzaron 18.43 req/s, mientras que en carga y resistencia se obtuvieron 7.60 y 6.13 req/s. Sin embargo, este resultado debe analizarse teniendo en cuenta el modelo de carga utilizado. La duración promedio de cada iteración fue cercana a 4.85 segundos, principalmente por los tiempos de espera (think time) de 1 a 4 segundos incluidos en el script.

Con este comportamiento, cada VU puede generar aproximadamente 0.41 solicitudes por segundo. Por esta razón, con 20 VUs se esperaría alrededor de 8.2 req/s y con 15 VUs cerca de 6.2 req/s, valores que son muy cercanos a los obtenidos. En la prueba de estrés se alcanzó el objetivo debido principalmente a que la concurrencia llegó a 100 VUs. Esto indica que la diferencia en throughput está relacionada con el modelo de carga y no necesariamente con una limitación de capacidad del Backend

6. Cuellos de botella y propuestas de mejora

Durante las pruebas no se identificó un cuello de botella evidente en el sistema. La latencia se mantuvo estable al aumentar la concurrencia hasta 100 VUs y la tasa de errores permaneció en 0%. El principal hallazgo corresponde al diseño del modelo de carga, ya que el SLO de throughput no fue ajustado teniendo en cuenta el think time utilizado.

Como primera mejora, se propone recalibrar el objetivo de throughput para cada tipo de prueba o modificar el think time si realmente se requiere alcanzar 15 req/s con 15 o 20 VUs. También sería conveniente ampliar la prueba de estrés por encima de 100 VUs para identificar el punto en el que realmente comienza a degradarse el servicio. Finalmente, en una siguiente ejecución se recomienda agregar métricas del lado servidor mediante APM, logs o métricas de infraestructura, ya que las mediciones actuales permiten observar principalmente el comportamiento del generador de carga.

7. Conclusión

En general, el sistema presentó un comportamiento estable durante las pruebas de carga, estrés y resistencia. Los objetivos de latencia y confiabilidad se cumplieron, sin errores y sin evidencias de degradación progresiva. La diferencia encontrada en el throughput se relaciona principalmente con el modelo de carga y los tiempos de espera definidos. Por esto, antes de utilizar este valor como criterio

formal de aceptación, sería conveniente ajustar el SLO o realizar una nueva prueba con un modelo de carga calibrado para el objetivo esperado.

Referencias

- Beyer, B., Murphy, N. R., Rensin, D. K., Kawahara, K., & Thorne, S. (Eds.). (2018). The site reliability workbook: Practical ways to implement SRE. O'Reilly Media.
- Gregg, B. (2021). Systems performance (2.a ed.). Addison-Wesley/Pearson.
- Grafana Labs. (s.f.). k6 documentation. Recuperado en 2026, de <https://k6.io/docs/>
- Molyneaux, I. (2015). The art of application performance testing (2.a ed.). O'Reilly Media.
- Universidad de La Salle. (2026). Recurso educativo digital 2: Pruebas y rendimiento de carga [Material del curso]. Calidad en Software II.